19001064 - Carlos Daniel Martinez Garcia
22003865 - Douglas Daniel Perez Hernandez

Delivery 3: Security Hardening (DevSecOps)

Task #3: Secret Protection (Pre-Commit Hook)

El objetivo de esta tarea fue implementar un mecanismo preventivo que impida la filtración accidental de información sensible hacia el repositorio de código.

En proyectos reales, uno de los riesgos más comunes no proviene de vulnerabilidades en librerías, sino de errores humanos, por ejemplo:

- Subir contraseñas al repositorio.
- Incluir API keys en archivos de configuración.
- Exponer tokens de autenticación.
- Guardar secretos directamente en el código fuente.

Cuando estos datos llegan a un repositorio remoto (especialmente si es público), pueden ser explotados inmediatamente por terceros.

Por lo tanto, se implementó un control preventivo en el flujo de desarrollo.

¿Qué es un Pre-Commit Hook?

Un **pre-commit hook** es un mecanismo automático que se ejecuta antes de que Git permita realizar un commit.

Esto permite:

- Analizar el contenido de los archivos modificados.
- Validar reglas de calidad.
- Bloquear commits que incumplan políticas establecidas.

En este caso, el hook fue diseñado para detectar posibles secretos antes de que el código sea almacenado en el historial del repositorio.

Herramienta Utilizada

Se utilizó la herramienta: **pre-commit**

Pre-commit permite configurar validaciones automáticas que se ejecutan antes de cada commit.

Implementación

Se configuró el archivo: **.pre-commit-config.yaml**

Con un hook local que analiza los archivos preparados para commit (staged files).

19001064 - Carlos Daniel Martinez Garcia
22003865 - Douglas Daniel Perez Hernandez

Configuración implementada:

```yaml
repos:
  - repo: local
    hooks:
      - id: prevent-secrets
        name: Prevent committing secrets
        entry: |
          powershell -Command "
          $files = git diff --cached --name-only;
          foreach ($file in $files) {
            if ($file -ne '.pre-commit-config.yaml' -and (Test-Path $file)) {
              if (Select-String -Path $file -Pattern 'API_KEY|SECRET|PASSWORD' -Quiet) {
                Write-Host '❌ Secret detected. Commit blocked.';
                exit 1;
              }
            }
          }
          "
        language: system
        pass_filenames: false
```

¿Cómo funciona el hook?

El proceso es el siguiente:

1. Git identifica los archivos que están listos para commit.
2. El hook obtiene la lista de archivos modificados.
3. Cada archivo es analizado en busca de patrones comunes asociados a secretos:
   - `API_KEY`
   - `SECRET`
   - `PASSWORD`
4. Si se encuentra alguna coincidencia:
   - Se muestra un mensaje de error.
   - El commit es bloqueado.
   - El archivo no se guarda en el historial del repositorio.

Este mecanismo actúa como una barrera preventiva en la fase más temprana del ciclo de desarrollo.

19001064 - Carlos Daniel Martinez Garcia
22003865 - Douglas Daniel Perez Hernandez

¿Cómo funciona el hook?

El proceso es el siguiente:

1. Git identifica los archivos que están listos para commit.
2. El hook obtiene la lista de archivos modificados.
3. Cada archivo es analizado en busca de patrones comunes asociados a secretos:
    - `API_KEY`
    - `SECRET`
    - `PASSWORD`
4. Si se encuentra alguna coincidencia:
    - Se muestra un mensaje de error.
    - El commit es bloqueado.
    - El archivo no se guarda en el historial del repositorio.

Este mecanismo actúa como una barrera preventiva en la fase más temprana del ciclo de desarrollo.

Prueba de Funcionamiento

Para validar la efectividad del mecanismo, se realizó la siguiente prueba:

Se creó un archivo con contenido: **SECRET=123456**

Al intentar ejecutar: **git commit -m "Testing secret protection"**

El sistema respondió: **Secret detected. Commit blocked.**

```
>> git commit -m "Fix hook to ignore config file"
Prevent committing secrets.....................................Passed
[deliv3p3 88fdf8f] Fix hook to ignore config file
 2 files changed, 17 insertions(+), 4 deletions(-)
 delete mode 100644 test.env
```

```
>> git add prueba.env
>> git commit -m "Testing secret protection"
Prevent committing secrets.....................................Failed
- hook id: prevent-secrets
- exit code: 1

❌ Secret detected. Commit blocked.
```

El commit fue bloqueado exitosamente.

Esto demuestra que el control funciona correctamente y evita la incorporación de información sensible en el repositorio.